



DEBRE TABOR UNIVERSITY

GAFAT INSTITUTE OF TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER
ENGINEERING(COMPUTER STREAM)

PROJECT TITLE: DESIGN AND IMPLEMENTATION OF A
RETRIEVAL-AUGMENTED LEGAL DOCUMENT ASSISTANT
USING A LARGE LANGUAGE MODEL

<u>Group members</u>	<u>ID</u>
1. Ayalew Awoke	DTU14R1593
2. Elsabet Demeke	DTU14R1043
3. Rediet Wassiyhun	DTU14R0827
4. Admasu Dessie	DTU14R1075

SUBMISSION DATE May, 2026

DEBRE TABOR, ETHIOPIA

DECLARATION

We hereby declare that this project entitled “*Design and Implementation of a Retrieval-Augmented Legal Document Assistant Using a Large Language Model*” is our own original work and has not been submitted to any other institution for the award of any degree or diploma.

All sources of information used in this project have been properly acknowledged and referenced. We confirm that this work was carried out by us in accordance with the academic rules and regulations of the institution.

No.	Name of Student	ID	Signature
1.	Ayalew Awoke	DTU14R1593	_____
2.	Elsabet Demeke	DTU14R1043	_____
3.	Rediet Wassyihun	DTU14R0827	_____
4.	Admasu Dessie	DTU14R1075	_____

Advisor name: Mr. Amlakie A.

Signature: _____

ABSTRACT

This thesis presents the design and implementation of an Amharic Legal Question Answering System based on Retrieval-Augmented Generation (RAG). In Ethiopia, accessing legal information remains a significant challenge due to the complexity of legal terminology and the scarcity of intelligent tools capable of processing the Amharic language. The primary objective of this work is to develop a system that can accurately interpret user queries posed in Amharic and retrieve context-based answers directly from legal documents. The proposed system integrates two core techniques: document retrieval and natural language generation, whereby legal texts are collected, preprocessed, and segmented into smaller chunks, subsequently transformed into vector embeddings and indexed in a vector database Chroma DB. Upon receiving a user query, the system retrieves the most semantically relevant document segments and supplies them to a large language model, which generates a coherent and concise response in Amharic. “The system is developed using Python, LangChain, Hugging Face Transformers, and ChromaDB within a Google Colaboratory environment. Experimental evaluation demonstrates that the system produces meaningful and contextually appropriate answers when relevant source documents are available; however, limitations persist in handling out-of-domain queries and achieving consistently high response accuracy. This work establishes the feasibility of combining retrieval and generation approaches to construct intelligent legal assistants for low-resource Amharic language, and identifies key directions for future research, including dataset expansion, multilingual support, and the adoption of more advanced language models.

ACKNOWLEDGEMENTS

First and foremost, we would like to express our sincere gratitude to our advisor, Mr. Amlakie, for his invaluable guidance, continuous support, and constructive feedback throughout the entire course of this thesis. His expertise, patience, and dedication were instrumental in shaping this work into its final form.

We would also like to extend our heartfelt appreciation to our friends, whose encouragement, motivation, and thoughtful discussions kept us focused and inspired during the most challenging phases of this project. Their willingness to listen and offer assistance whenever needed made a meaningful difference in the completion of this work.

Finally, our deepest gratitude goes to our families, whose unconditional love, endless patience, and unwavering support have been our greatest source of strength throughout our academic journey. This achievement is as much theirs as it is ours.

Table of Contents

page

DECLARATION	I
ABSTRACT.....	II
ACKNOWLEDGEMENTS	III
LIST OF FIGURES	VII
ACRONYMS AND ABBREVIATIONS	VIII
CHAPTER ONE	1
INTRODUCTION	1
1.1 Introduction	1
1.2 Background of the Study.....	1
1.3 Problem Statement	2
1.4 Objectives of the Study	3
1.4.1 General Objective	3
1.4.2 Specific Objectives	3
1.5 Significance of the Study	3
1.6 Limitations of the Study	4
1.7 Scope of the Study.....	4
CHAPTER TWO	5
LITERATURE REVIEW	5
2.1 Introduction	5
2.2 Related Works	6
2.3 Gap Identification.....	6
CHAPTER THREE	8
METHODOLOGY	8
3.1 Introduction	8

3.2 Model Building Methodology	8
3.3 System Architecture	9
3.4 Data Collection.....	10
3.5 Data Preprocessing.....	10
3.6 RAG Design Pipeline	11
3.6.1 Text Chunking	12
3.6.2 Embedding and Indexing.....	13
3.6.3 Query Processing.....	14
3.6.4 Retrieval Mechanism	14
3.6.5 Response Generation	15
3.6.6 System Integration.....	15
3.6.7 Evaluation Metrics.....	16
3.7 Tools and Libraries.....	16
3.8 Experimental Environment	17
CHAPTER FOUR.....	18
RESULT AND DISCUSSION	18
4.1 Introduction	18
4.2 Chunking Result.....	18
4.3 Embedding and Vector Indexing Result	19
4.4 Model Building	20
4.5 Testing and Evaluation.....	23
CHAPTER FIVE	26
CONCLUSION AND RECOMMENDATION.....	26
5.1 Conclusion.....	26
5.2 Recommendation.....	26

References 28

Appendces 28

Appendix A: User registration functionality 28

Appendix B: Response generation functionality..... 30

LIST OF FIGURES

Figure 1 System Architecture	9
Figure 2 Working of RAG and Pipeline	12
Figure 3 Sample chunked text.....	18
Figure 4 Sample Indexed vector	19
Figure 5 Sign Up Page	21
Figure 6 Chatbot Interface	21
Figure 7 System Response When Relevant Data is Found.....	22
Figure 8 System Response When No Relevant Information is Found	22
Figure 9 Evaluation Dataset.....	23
Figure 10 Offline Evaluation Metrics Bar Chart	24

ACRONYMS AND ABBREVIATIONS

Acronym	Full Form
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BLEU	Bilingual Evaluation Understudy
BM25	Best Matching 25
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DB	Database
FAISS	Facebook AI Similarity Search
FDRE	Federal Democratic Republic of Ethiopia
GB	Gigabyte
GPT	Generative Pre-trained Transformer
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
LLMs	Large Language Models
NLP	Natural Language Processing
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
REST	Representational State Transfer
ROUGE-L	Recall-Oriented Understudy for Gisting Evaluation
RRF	Reciprocal Rank Fusion

CHAPTER ONE

INTRODUCTION

1.1 Introduction

The rapid advancement of Artificial Intelligence (AI) and Natural Language Processing (NLP) has opened new possibilities for developing intelligent systems capable of understanding and responding to human language. Among the most promising developments in this area is the emergence of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG), which together enable systems to retrieve relevant information from documents and generate accurate, context-based responses in natural language.

In Ethiopia, legal information plays a critical role in the daily lives of citizens, yet it remains largely inaccessible to the majority of the population. Legal documents are typically written in complex, formal language, and there is a significant lack of intelligent tools that support the Amharic language for legal information retrieval. This gap motivated the development of the present system, which aims to bridge the divide between complex legal texts and Amharic-speaking users by leveraging modern AI techniques.

This thesis presents the design and implementation of an Amharic Legal Question Answering System that uses Retrieval-Augmented Generation to retrieve and generate answers from Ethiopian legal documents. The system is accessible through a web-based chatbot interface, allowing users to submit legal queries in Amharic and receive contextually relevant responses.

1.2 Background of the Study

In Ethiopia, accessing legal information is often a significant challenge due to the complexity of legal documents and the limited availability of intelligent systems that support the Amharic language. Most legal resources are written in formal and technical language, making it difficult for students, legal professionals, and the general public to easily understand and retrieve relevant information in a timely and efficient manner.

With the rapid advancement of Artificial Intelligence, particularly in the domain of Large Language Models, it has become increasingly feasible to develop systems capable of understanding natural language and providing meaningful, context-aware responses. One of the most prominent approaches in this regard is Retrieval-Augmented Generation, a technique that combines document retrieval with natural language generation to produce accurate and contextually grounded answers derived directly from source documents.

This project focuses on the design and implementation of an Amharic Legal Question Answering System using the RAG framework. The system is designed to retrieve relevant segments from Ethiopian legal documents and generate precise answers in Amharic, thereby enabling users to access legal knowledge more efficiently and in a language they are most comfortable with.

1.3 Problem Statement

Despite the existence of a substantial body of legal documents in Ethiopia, there is a notable absence of intelligent systems that allow users to query legal information in Amharic and receive clear, accurate, and contextually relevant answers. Users who seek specific legal information are frequently confronted with lengthy, technically worded documents that require considerable time and expertise to navigate.

The absence of Amharic-language search and question answering tools means that a large portion of the population remains effectively excluded from accessing legal knowledge that directly affects their rights and responsibilities. Furthermore, the scarcity of NLP tools tailored to the Amharic language compounds this problem, as existing general-purpose systems fail to adequately address the linguistic and contextual nuances of Ethiopian legal texts. As a result, accessing legal knowledge becomes time-consuming, inefficient, and inaccessible to the majority of users. This project therefore aims to address these challenges by designing and implementing a system capable of retrieving and generating accurate answers from legal documents based on user queries submitted in Amharic.

1.4 Objectives of the Study

1.4.1 General Objective

To design and implement a Retrieval-Augmented Legal Document Assistant using a Large Language Model for the Amharic language.

1.4.2 Specific Objectives

In order to achieve the general objective, the following specific objectives are pursued.

- ✓ To collect and prepare a set of Ethiopian legal documents suitable for use in the system.
- ✓ To preprocess and structure the collected legal text data into a format appropriate for retrieval.
- ✓ To integrate a large language model capable of generating answers in Amharic based on retrieved context.
- ✓ To evaluate the performance of the system using a set of sample legal queries.

1.5 Significance of the Study

This project carries considerable significance in both the technical and societal dimensions. From a technical standpoint, it demonstrates the practical applicability of modern AI techniques, specifically Large Language Models and Retrieval-Augmented Generation, to the domain of legal information retrieval in a low-resource language context. From a societal perspective, the system has the potential to make legal knowledge more accessible to students, researchers, legal practitioners, and members of the general public who may otherwise lack the means to navigate complex legal texts.

Beyond its immediate application, this work contributes to the broader advancement of Amharic Natural Language Processing by providing a concrete implementation that future researchers may build upon. It also serves as a foundation for the development of more comprehensive intelligent legal assistant systems tailored to the Ethiopian context, ultimately contributing to the democratization of legal information access in the country.

1.6 Limitations of the Study

This project is subject to several limitations that are important to acknowledge. First, the system is designed exclusively for the Amharic language and does not support other Ethiopian languages or English-language queries, which restricts its applicability to Amharic-speaking users only. Second, while the system is equipped with a legal document database, this database is not fully comprehensive and covers only a limited subset of Ethiopian legal texts, meaning that queries falling outside the available documents may not receive accurate or complete answers. Third, the system relies entirely on the GROQ API for language model inference and therefore cannot function in an offline environment, as no locally deployable large language model with sufficient Amharic language support is currently available. Fourth, the absence of a large-scale, annotated Amharic legal dataset significantly constrains the system's ability to handle the full diversity of legal queries and limits the depth of evaluation that could be performed. Fifth, the system is implemented as a research prototype accessible through a web-based chatbot interface and has not been deployed in a real-world legal setting, which means its performance under practical conditions remains to be fully validated.

1.7 Scope of the Study

This project is concerned with the development of a prototype system designed to answer legal questions in Amharic using a selected set of Ethiopian legal documents. The system operates on a limited dataset comprising specific areas of Ethiopian law, including criminal law and bail law, and is implemented within a Google Colaboratory computational environment. The core methodology is based on the Retrieval-Augmented Generation framework, and the system is designed to produce short, context-based answers in response to user queries submitted through a web-based chatbot interface.

It is important to note that this project does not aim to cover all domains of Ethiopian law, nor does it extend to full-scale deployment or the provision of real-time legal advisory services. The system is intended as a research prototype demonstrating the applicability of RAG-based approaches to legal question answering in a low-resource language setting.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

The rapid advancement of Artificial Intelligence (AI) and Natural Language Processing (NLP) has transformed information retrieval and conversational systems. Traditional chatbot systems relied on rule-based approaches using predefined rules and decision trees, which lacked contextual understanding when processing complex queries, particularly in specialized domains such as legal information systems [1]. This chapter reviews literature related to the development of an Amharic Retrieval-Augmented Generation (RAG) Based Legal Document Assistant, examining key concepts including NLP, Large Language Models (LLMs), semantic search, vector databases, and RAG frameworks, while identifying research gaps that justify the proposed system.

Recent transformer-based architectures such as BERT, GPT, T5, and LLaMA have significantly improved natural language understanding and text generation. These models learn contextual relationships between words and sentences, enabling more accurate and context-aware responses [2], and assist users in retrieving, summarizing, and answering legal questions more effectively than traditional keyword-based methods. Building on this, Retrieval-Augmented Generation (RAG) has emerged as one of the most effective techniques for improving factual reliability by combining document retrieval with generative models before generating responses [3], reducing hallucination and ensuring responses are grounded in verified information — which is critical in high-accuracy domains like legal systems.

The effectiveness of RAG systems depends heavily on embedding models and vector databases. Embedding models transform text into dense vector representations that capture semantic meaning, improving retrieval even when query wording differs from document text [4]. Vector databases such as ChromaDB, FAISS, and Pinecone efficiently store and retrieve these embeddings, with ChromaDB gaining popularity for its scalability and semantic retrieval capabilities. Additionally, preprocessing strategies like tokenization, chunking, and normalization improve retrieval precision by breaking lengthy legal documents into semantically meaningful sections [5].

Despite advances in multilingual NLP through models like XLM-RoBERTa and mBERT [6], low-resource languages such as Amharic remain underserved due to limited datasets and domain-specific pretrained models. In the Ethiopian legal context, legal documents are complex and inaccessible to non-legal professionals, and existing systems rely on keyword-based search that fails to capture semantic relevance. This creates a clear need for an intelligent Amharic legal assistant grounded in semantic retrieval and LLMs— the gap this system directly addresses.

2.2 Related Works

ChatGPT is an AI-powered conversational assistant built on Large Language Models, capable of generating human-like responses across domains including question answering, summarization, and conversational interaction using transformer-based architectures. However, ChatGPT is a general-purpose model and is not optimized for Ethiopian legal documents or Amharic legal information retrieval, making it insufficient for the specialized needs addressed by this project. Although ChatGPT demonstrates strong general conversational capabilities, it lacks domain-specific optimization for Ethiopian legal document retrieval in Amharic. [6].

LegalZoom is an online legal assistance platform that provides legal documents, consultations, and digital legal guidance, helping users prepare legal forms and access legal services. Despite its capabilities, LegalZoom primarily targets English-speaking users and does not support Amharic legal document retrieval or Retrieval-Augmented Generation techniques tailored to the Ethiopian legal context, limiting its applicability for Amharic-speaking users seeking legal assistance [7].

M-Wakili is an AI-powered legal assistant developed to provide legal support and awareness in African contexts, offering legal guidance and information to users across the continent. While M-Wakili represents a relevant step toward AI-driven legal assistance in Africa, the proposed system distinguishes itself by integrating Retrieval-Augmented Generation, vector databases, semantic embeddings, and Amharic legal document retrieval specifically designed and tailored for the Ethiopian legal system [8].

2.3 Gap Identification

The reviewed literature demonstrates significant progress in Artificial Intelligence, Natural Language Processing, Large Language Models, and Retrieval-Augmented Generation systems.

Existing AI-powered legal assistants have improved access to legal information through conversational interfaces and automated question answering systems, while semantic search and vector databases have enhanced the retrieval of relevant documents well beyond the capabilities of traditional keyword-matching techniques.

Despite these advancements, several important limitations remain. Most existing legal assistant systems are developed primarily for high-resource languages such as English and are not optimized for low-resource languages like Amharic. Furthermore, many of these systems continue to rely on traditional keyword-based search methods, which often fail to capture the semantic meaning and contextual relationships present within complex legal queries, resulting in incomplete or inaccurate responses.

The proposed system seeks to address these identified gaps by integrating semantic embeddings, ChromaDB vector databases, multilingual transformer models, and Retrieval-Augmented Generation techniques to provide accurate and context-aware legal assistance in the Amharic language. The system aims to improve accessibility to Ethiopian legal information and provide users with intelligent legal document retrieval capabilities tailored specifically to the Ethiopian legal environment.

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter presents the methodology adopted in the design and implementation of the Amharic Retrieval-Augmented Legal Document Assistant using a Large Language Model. The chapter describes the system development methodology, system architecture, advanced RAG pipeline design, Experimental requirements conducted throughout the project. The purpose of this methodology is to provide a systematic and well-structured framework for designing, developing, and evaluating an intelligent legal assistant capable of understanding and responding to legal queries in the Amharic language.

The proposed system integrates Artificial Intelligence, Natural Language Processing, Retrieval-Augmented Generation, vector databases, semantic search, dense and sparse retrieval mechanisms, and Large Language Models to improve accessibility to Ethiopian legal information. Since legal documents are often large, complex, and difficult for ordinary citizens to interpret, the proposed system aims to simplify legal information retrieval through a web-based chatbot interface and advanced semantic understanding of Amharic legal queries.

3.2 Model Building Methodology

The model building methodology adopted for this project is the Agile Software Development Methodology. Agile is an iterative and incremental software development approach that supports continuous development, testing, evaluation, and refinement throughout the system development lifecycle. This methodology was selected because the proposed system integrates multiple advanced technologies such as Natural Language Processing, semantic retrieval, dense and sparse vector search, embeddings, and Large Language Models, which require continuous experimentation and improvement during implementation. Unlike traditional development approaches, Agile methodology allows system components to be developed and evaluated progressively, making it easier to identify implementation challenges and improve system performance at each stage of development.

3.3 System Architecture

The proposed system follows a layered architecture that integrates a web-based frontend chatbot interface, a backend processing engine, an advanced semantic retrieval module combining dense, sparse, and hybrid search mechanisms, a vector database, and a Large Language Model for response generation. The frontend interface is built using HTML, CSS, and JavaScript and provides users with a simple, responsive, and Amharic-compatible conversational interface through which legal queries can be submitted and responses displayed in real time. The backend is developed using Python and FastAPI, which handles all server-side processing including query handling, embedding generation, retrieval coordination, prompt construction, and response management.

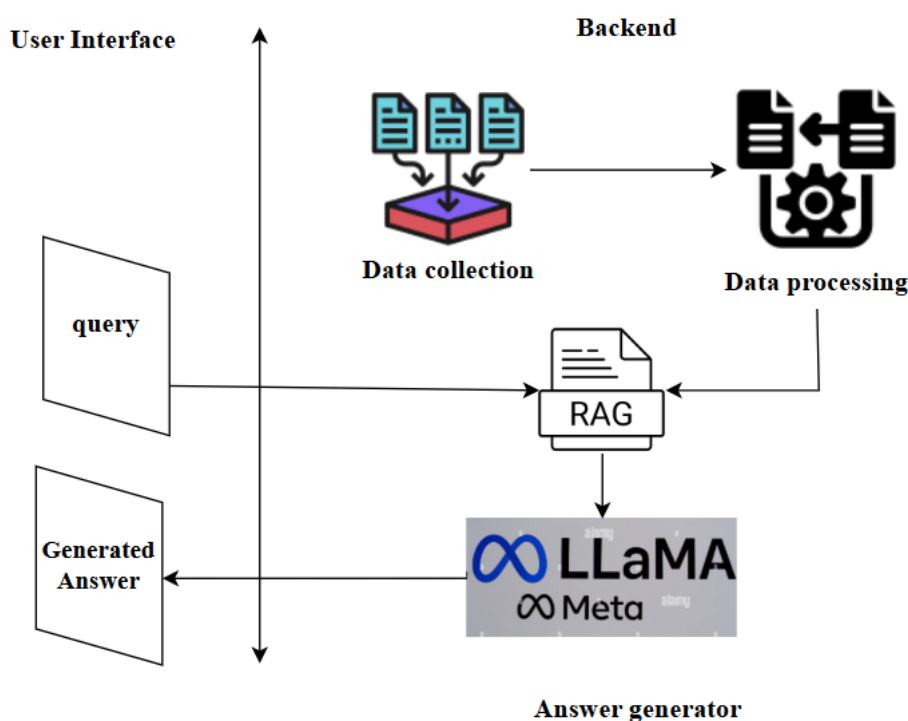


Figure 1 System Architecture

When a user submits a legal query, the backend converts it into both a dense vector embedding using a multilingual sentence-transformer model and a sparse BM25 representation simultaneously. These two retrieval signals are combined through a hybrid retrieval mechanism using Reciprocal Rank Fusion to produce a unified ranking of the most relevant legal document chunks, which are then passed to the Large Language Model through the GROQ API to generate a contextually grounded response in Amharic and returned to the user through the chatbot interface.

3.4 Data Collection

Ethiopian legal documents were collected from publicly available and reliable sources, including constitutional documents, proclamations, criminal laws, bail laws, and civil law references written in the Amharic language. The documents were obtained from three primary online repositories: [9] the Ministry of Justice of the Federal Democratic Republic of Ethiopia website, [10] Abyssinia Law's civil law forms page, and [11] 5A Law Firm's codes page. A total of 39 Amharic legal documents were collected in multiple file formats, including PDF and other structured text formats, with a combined storage size of 208 MB, serving as the primary knowledge base for the retrieval system. Special care was taken to ensure that only authentic and verified legal texts were included in the dataset in order to maintain the accuracy and reliability of the information provided by the system. The collected documents were stored in structured directories organized by legal domain to facilitate efficient preprocessing and indexing. This data collection work was conducted at Debre Tabor University.

3.5 Data Preprocessing

Following data collection, the legal documents underwent a comprehensive preprocessing pipeline tailored specifically for Amharic language content. Raw text was extracted from PDF and unstructured formats, with careful handling of Ethiopic script encoding issues common in digitally scanned or poorly formatted legal documents. Amharic-specific noise removal then eliminated unnecessary symbols, non-Ethiopic characters outside the Unicode range U+1200–U+137F, irregularly mixed Arabic numerals, HTML artifacts, and formatting remnants introduced during extraction. Final cleaning ensured whitespace normalization, removal of stray line breaks, and

standardization of legal numbering formats to produce uniform document structure before chunking and embedding.

A critical step in the pipeline was Amharic Unicode normalization, which standardized Ethiopic characters sharing the same phonetic value but differing in Unicode codepoint. Several character families were mapped to single canonical forms: the ሐ and ኀ families were normalized to the ሀ family; and the ዐ family to አ. Additional families fully normalized include the ሀ (Ha), ሰ (Se), ጸ (Tse), and ገ (Ge) families, each consolidated to one representative form for consistent embedding and retrieval across document variations. Crucially, the same normalization was applied symmetrically to user input queries at retrieval time, ensuring that character-level variation in queries does not produce mismatches against the normalized corpus — thereby improving overall retrieval accuracy and system robustness.

3.6 RAG Design Pipeline

RAG (Retrieval-Augmented Generation) is an AI framework that combines information retrieval with language model generation to produce accurate and grounded responses. It works by first retrieving relevant documents or text chunks from an external knowledge base based on the user's query. These retrieved results are then given to the language model as additional context. The model uses both the query and retrieved information to generate a final answer. This approach reduces hallucination by ensuring responses are based on real data rather than model memory alone. It is especially useful in domains like law where factual accuracy is critical. Overall, RAG improves the reliability and precision of AI systems by linking search with generation.

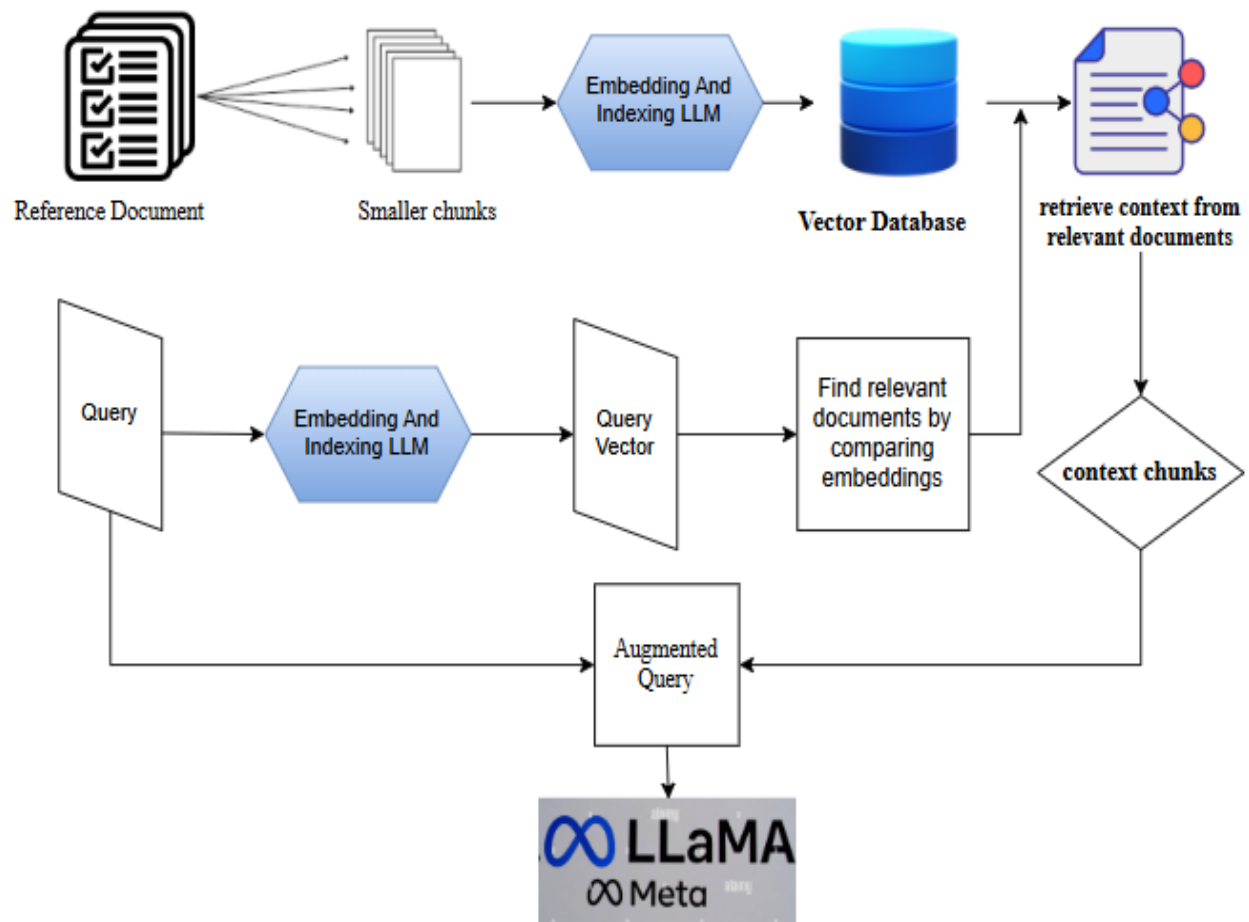


Figure 2 Working of RAG and pipeline

3.6.1 Text Chunking

Legal documents were segmented into meaningful units using a hierarchical chunking strategy that combines two complementary boundary detection mechanisms operating in a defined priority order. The primary mechanism employs a unified heading recognition pattern that detects structural markers exclusively at the beginning of new lines, preventing body text occurrences from being mistakenly treated as boundaries. These markers follow a strict priority hierarchy: **ጥዕረ-ፍ** (chapter) as the broadest division, followed by **ክፍል** (part), **ንዑስ ክፍል** (sub-part), **አንቀጽ** (article), **ንዑስ አንቀጽ** (sub-article), and **አዋጅ** (proclamation) as a document-level boundary. Each marker is recognized across all numeric representations, including Arabic numerals, Ethiopic numerals (**፩፡፪፡፫**), and Amharic cardinal words (**አንድ** through **መቶ**), ensuring robust detection

across the full diversity of Ethiopian legal document formatting. When accumulated text under a detected heading exceeds the maximum chunk size of 1,000 characters before a subsequent heading is encountered, the chunk is finalized at the nearest full stop boundary, preserving semantic completeness at the sentence level. For documents entirely lacking structural headings, this sentence-boundary mechanism serves as the sole segmentation strategy, applied with a chunk overlap of 150 characters to maintain contextual continuity across consecutive chunks.

This two-tier design ensures that both broad legal context and fine-grained provision-level meaning are represented at appropriate granularity, substantially improving retrieval precision over arbitrary splitting methods. The strict priority ordering of heading recognition guarantees that higher-level structural divisions consistently supersede lower-level ones, faithfully preserving the natural legal hierarchy of Ethiopian legislative texts. Each resulting chunk was enriched with structured metadata comprising the source document name, article number, chapter or title reference, and legal domain, enabling precise retrieval and accurate citation generation during system response.

3.6.2 Embedding and Indexing

After preprocessing, the cleaned and chunked legal document segments were converted into dense vector representations using a multilingual sentence-transformer embedding model. These dense embeddings capture deep semantic meaning and contextual relationships of the legal texts, enabling the system to retrieve relevant information based on meaning rather than surface-level keyword matching. The generated dense embeddings were subsequently indexed and stored within ChromaDB, which serves as the primary vector database for the system. ChromaDB was selected due to its efficient semantic retrieval capabilities, scalability, and strong compatibility with transformer-based embedding models.

In addition to dense embeddings, a sparse index was constructed in parallel using BM25, a probabilistic term-based ranking algorithm widely used in information retrieval. The BM25 sparse index captures exact lexical matches between query terms and document terms, complementing the semantic coverage of the dense embeddings. The combination of dense vector indexing in ChromaDB and sparse BM25 indexing forms the dual-index structure that supports the advanced hybrid retrieval mechanism of the proposed system.

3.6.3 Query Processing

Before response generation and retrieval, every user query undergoes a structured processing pipeline designed to ensure semantic alignment with the normalized document corpus. The query is first subjected to the same Amharic Unicode normalization applied during document preprocessing, mapping phonetically equivalent Ethiopic character families to their canonical forms and eliminating any irrelevant noise, as character-level mismatches between the query and indexed chunks would directly degrade retrieval accuracy. The normalized query is then converted into a dense vector representation using the same embedding model applied during document indexing, after which cosine similarity search is performed within the ChromaDB vector database to identify semantically relevant document chunks. Simultaneously, the normalized query is processed by the BM25 sparse retrieval component, matching query terms against the document collection based on term frequency and distribution.

3.6.4 Retrieval Mechanism

The retrieval mechanism of the proposed system is built upon a hybrid retrieval architecture that combines dense semantic search, sparse keyword-based search, and a neural reranking stage to maximize retrieval accuracy and coverage across diverse legal queries. This multi-strategy approach was adopted because legal documents contain both highly specific legal terminology that benefits from exact keyword matching and complex contextual meaning that requires semantic understanding to retrieve accurately. The dense retrieval component operates by converting the user query into a dense vector embedding and performing cosine similarity search within the ChromaDB vector database to identify the most semantically similar legal document chunks, contributing 30 percent of the candidate pool. This component is particularly effective for queries that use paraphrased or semantically equivalent language that may differ in wording from the original legal texts. In parallel, the sparse retrieval component employs the BM25 algorithm, which scores document chunks based on the frequency and distribution of query terms within the document collection, equally contributing 30 percent of the candidate pool. This component is particularly effective for queries containing specific legal terms, article numbers, or exact phrases that appear directly within the legal documents.

The candidate results from both retrieval components are subsequently merged using Reciprocal Rank Fusion, a rank aggregation technique that combines ranked lists from multiple retrieval sources by assigning scores based on the reciprocal of each document rank position in each list, ensuring that chunks highly relevant according to both strategies receive the highest combined scores. The fused candidate set is then passed through a dedicated neural reranking stage, which contributes the remaining 40 percent of retrieval influence by re-scoring each candidate chunk against the original query using deep cross-attention, capturing fine-grained semantic relevance that initial retrieval stages may overlook. This reranking stage ensures that the final ranked list presented to the response generation component represents the most contextually precise and legally relevant document chunks, making the overall retrieval pipeline robust against both vocabulary mismatch and semantic ambiguity inherent in Ethiopian legal language.

3.6.5 Response Generation

The response generation component of the system is powered by a Large Language Model accessed through the GROQ API. Once the most relevant legal document chunks have been retrieved through the hybrid retrieval mechanism, they are combined with the original user query and structured into a carefully engineered prompt that instructs the language model to generate a response grounded exclusively in the provided legal context. The prompt engineering strategy was designed to minimize hallucination, enforce contextual faithfulness, and ensure that responses are generated in the Amharic language with appropriate legal clarity and precision. The language model processes the structured prompt and generates a coherent, accurate, and contextually grounded response that is directly supported by the retrieved legal documents.

3.6.6 System Integration

The complete advanced RAG pipeline was integrated into a unified end-to-end system by combining the frontend chatbot interface, the backend processing engine, the hybrid retrieval module, the ChromaDB vector database, the BM25 sparse index, and the GROQ-powered Large Language Model. The frontend communicates with the backend through RESTful APIs developed using FastAPI, enabling real-time query submission and response display. The backend coordinates all processing steps from query reception through dense and sparse embedding generation, hybrid retrieval, reciprocal rank fusion, prompt construction, and response generation,

returning the final answer to the frontend for display to the user. The chatbot interface was designed to correctly render Amharic Unicode characters and support natural conversational interaction, providing a seamless and intuitive user experience throughout the system.

3.6.7 Evaluation Metrics

Four evaluation metrics were used to assess the performance of the system.

BLEU (Bilingual Evaluation Understudy) measures the overlap of n-grams (word sequences) between the system-generated answer and a reference answer. A score closer to 1.0 indicates that the output closely matches the reference word-for-word, while a score near 0 indicates little to no match.

ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation) measures the longest common subsequence between the generated and reference texts, capturing how well the system preserves the order and flow of words compared to the expected answer.

Semantic Similarity measures how close the meaning of the generated response is to the reference answer using vector embeddings rather than exact word matching, where a score near 1.0 means the two responses carry nearly the same meaning even if worded differently.

Context Relevancy measures how relevant the retrieved document chunks are to the user's query, where a higher score indicates more appropriate legal content retrieval. Together, the four metrics cover two key dimensions of system quality: generation quality measured by BLEU, ROUGE-L, and Semantic Similarity, and retrieval quality measured by Context Relevancy. Since no existing annotated Amharic legal evaluation dataset is currently available, a dataset of ten sample Amharic legal questions with their corresponding reference answers was manually prepared in CSV format and used to compute the four evaluation metric scores.

3.7 Tools and Libraries

The system was developed using a carefully selected set of tools and libraries suited to the requirements of an advanced RAG-based legal assistant. LangChain was used as the primary framework for orchestrating the RAG pipeline, managing prompt construction, and coordinating interactions between the retrieval and generation components. Hugging Face Transformers

provided the multilingual sentence-transformer embedding models used for generating dense vector representations of legal document chunks and user queries. ChromaDB was used as the vector database for storing and retrieving dense document embeddings through semantic similarity search. The BM25 algorithm was implemented using the rank-bm25 Python library to construct the sparse index and perform keyword-based retrieval in parallel with the dense search. Reciprocal rank fusion was implemented as a custom ranking module within the backend pipeline to merge results from both retrieval components. The GROQ API was used to access the Large Language Model responsible for generating responses in Amharic. FastAPI was used for developing the backend RESTful APIs, and the frontend chatbot interface was implemented using HTML, CSS, and JavaScript to ensure compatibility and responsiveness across different devices and browsers.

3.8 Experimental Environment

Python was selected as the primary programming language for the backend development of the system due to its extensive support for artificial intelligence, machine learning, and natural language processing frameworks, as well as its comprehensive library ecosystem and ease of integration with tools such as LangChain, Hugging Face Transformers, ChromaDB, rank-bm25, and FastAPI. The frontend of the system was developed using HTML, CSS, and JavaScript, providing the necessary foundation for building a responsive and user-friendly chatbot interface with proper Amharic Unicode character rendering and real-time API communication.

The system was developed and tested within a cloud-based computational environment utilizing Google Colaboratory and Kaggle notebooks. Google Colaboratory provided up to 12.7 GB of RAM alongside GPU acceleration, while Kaggle notebooks offered up to 30 GB of RAM with additional GPU support, together providing sufficient processing power for embedding generation, vector indexing, BM25 index construction, and retrieval pipeline execution without requiring dedicated high-performance local hardware. For deployment, the system requires a server or cloud instance with sufficient memory to load the multilingual sentence-transformer embedding models, maintain the ChromaDB vector index, and handle concurrent user requests efficiently. A stable internet connection is additionally required at runtime since the system relies on the GROQ API for language model inference, as locally deployable large language models with adequate Amharic language support are not currently available.

CHAPTER FOUR

RESULT AND DISCUSSION

4.1 Introduction

This chapter presents the results and discussion of the RAG-based Amharic Legal Document Assistant across four successive stages of the system pipeline. It covers the outcomes of text chunking, document embedding and vector indexing, document retrieval performance demonstrated through sample legal queries, and the final end-to-end system evaluation following full integration with the frontend interface. Each stage is presented systematically to provide a comprehensive understanding of how the pipeline components collectively contribute to the overall system performance in delivering accurate and legally grounded responses to Amharic-speaking users.

4.2 Chunking Result

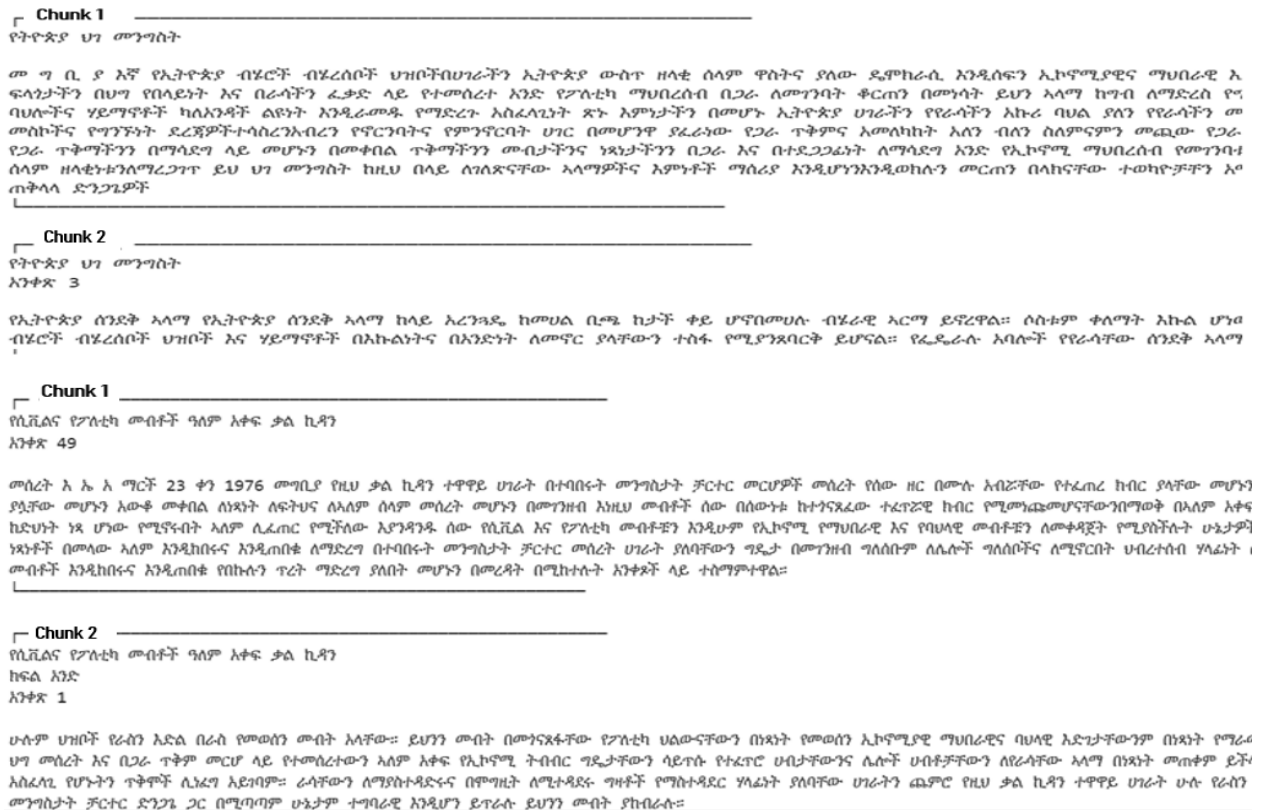


Figure 3 Sample chunked text

The figure 3 above presents sample outputs of the hierarchical chunking pipeline applied to Amharic legal documents, where each chunk is clearly delimited and labeled with its corresponding structural marker such as **ጥዕራፍ** (chapter), **ክፍል** (part), and **አንቀጽ** (article). The pipeline successfully processed the entire legal document corpus, producing a total of 2,345 semantically complete and structurally coherent chunks, demonstrating that the heading recognition pattern effectively identified legal boundaries and generated meaningful units ready for embedding and vector indexing.

4.3 Embedding and Vector Indexing Result

TEXT:

የኢትዮጵያ የፍትሕ ብሔር ሥነ ሥርዓት ስግ

ይህ ህግ በሰራ ላይ አንዲውል ከታወጀበት ጊዜ ጀምሮ ለሚቀርቡ ከሶትና በከርከር ላይ ላሉትም ከሶት ተፈጻሚ ይሆናል። ከሰን ተቀብሎ መስማት የጀመረው ፍርድ ቤት በዚህ ህግ ያልተጠቀሰ ወይም የተሰረዘ ቢሆንም አንኳ በቀድሞው ህግ መሰረት በነበረው ስልጣኑ የጀመረውን ነገር መፈጸም አለበት። የዚህ ህግ ድንጋጌዎች በሰራ ላይ ከመዋላቸው በፊት በይርጋ የታገዱትን ጉዳዮች ይህን ህግ መሰረት በማድረግ ባለፈው ነገር ለማንቀሳቀስ አይቻልም። በዚህ ህግ መሰረት የሚጻፍው የይርጋ ዘመን የሚቆጠረው ህጉ በሰራ ላይ አንዲውል ከታወጀበት ቀን ጀምሮ ነው። የዚህ ህግ ድንጋጌዎች የይርጋ ዘመን የሚጻፍበትን ጊዜ የሚያረዝሙ ሲሆን የከ በተወሰነው መሰረት ተፈጻሚ ይሆናል። ይህም ሲሆን ህጉ ከመጽናኑ በፊት ያለፈው ጊዜ ከዋናው የይርጋ ዘመን ላይ መቀነስ አለበት። ይህ ህግ የይርጋ ዘመን ወሰን የሚያሳየር ሲሆን ህጉ ከመጽናኑ በፊት ያለፈው ዘመን ተወስኖ የነበረው የይርጋ ዘመን ተፈጻሚ ይሆናል። በዚህ ህግ በልዩ አኳኋን አንዲተረጎሙ በግልጽ የተመለከተ ድንጋጌ ከሌለ በቀር ቀጥሎ ያሉት ቃላቶችና አረፍተ ነገሮች ከዚህ በታች አንድተመለከተው ይተረጎማሉ።

VECTOR:

0.006411	0.047953	-0.026096	0.009602	0.013166	-0.059319	-0.008937	-0.027653	0.002401	0.053649	-0.065117	0.010083	0.112141	0.037015	-0.045407
-0.055452	0.045177	-0.017352	0.062516	-0.019499	0.003622	-0.027996	0.006160	0.006911	-0.002769	-0.000523	0.020304	0.042671	-0.025885	0.027895
0.006701	-0.033509	-0.009152	0.012204	0.022406	0.021034	-0.032015	-0.015228	-0.000532	0.013377	0.008387	0.028671	0.008688	-0.071322	-0.004391
-0.003177	0.016082	0.032234	-0.022208	-0.015370	0.032313	-0.002861	0.018977	0.010421	-0.077016	-0.062973	0.027274	0.047604	-0.035612	-0.056109
-0.031611	0.037684	-0.008157	-0.006533	0.036522	-0.033909	0.006591	-0.025376	-0.023229	-0.013317	-0.007039	-0.028455	0.081107	-0.016149	-0.029119
0.017993	-0.0083205	0.012761	-0.004964	-0.036268	0.040077	-0.000414	-0.006011	0.066221	0.036653	-0.024832	-0.010183	0.062679	0.013994	0.017639
0.018134	0.008498	-0.022473	0.047312	0.035444	0.016781	-0.001692	0.013188	0.026609	-0.028732	-0.017834	-0.077631	-0.052019	-0.069626	-0.073495
0.019889	-0.021717	-0.039309	0.044242	-0.048387	-0.000547	0.029520	0.011445	-0.075171	-0.004618	-0.027785	0.034613	-0.024660	-0.004232	-0.028437
-0.030432	0.031684	0.008436	-0.050288	0.038998	-0.006826	-0.002033	-0.021533	0.027616	-0.064082	-0.023679	-0.048935	-0.006059	-0.022862	-0.012823
0.030526	0.041209	0.049534	0.003621	0.010883	0.011224	-0.072860	-0.016561	0.039483	0.045272	-0.0083289	0.049157	-0.003215	-0.017459	0.036504
0.063930	-0.038431	-0.006780	0.002471	0.001808	-0.000833	-0.037996	-0.020908	-0.032425	0.022305	0.044932	0.005507	0.030070	-0.010142	0.050779
-0.015088	-0.022928	-0.006284	-0.035286	-0.016096	-0.014702	-0.031059	-0.047656	0.037469	0.017817	-0.045411	-0.010616	-0.004926	0.007212	-0.074456
-0.019206	-0.031891	-0.035475	0.028153	0.006120	-0.015265	-0.026295	-0.045063	0.019205	0.016001	-0.016525	0.065167	0.044613	0.009705	0.014576
0.056710	0.045072	0.024146	-0.018809	0.006601	0.027856	-0.020010	0.011648	0.025765	0.028783	-0.083157	0.051477	0.032571	0.051424	0.032198
-0.026558	0.044518	-0.052474	0.013592	-0.019026	-0.049601	0.013569	0.038319	-0.065448	0.019624	0.040128	-0.079737	0.075882	0.007814	-0.009948
0.002877	0.022517	0.010060	0.017184	0.030393	0.083936	0.079393	-0.040099	0.023980	-0.004945	-0.020631	0.021822	-0.002404	-0.012024	-0.090490
-0.042461	0.061526	0.033008	-0.032365	0.042966	-0.032075	-0.047576	0.049014	-0.061857	-0.030274	0.010554	-0.011448	0.020377	-0.004265	0.008017
0.023676	-0.021391	-0.024614	0.014719	-0.005165	0.012103	-0.072431	-0.033634	-0.052332	-0.007015	-0.009559	0.051340	-0.083924	-0.022730	-0.002580
-0.005588	-0.013938	-0.014836	0.010551	0.017525	-0.000979	0.017679	-0.005684	-0.020714	-0.028196	-0.030668	0.089307	-0.087042	-0.016914	-0.043340

Figure 4 Sample Indexed vector

The figure 4 above illustrates the embedding and vector indexing result of a sample Amharic legal document chunk, displaying both the original Amharic legal text and its corresponding dense vector representation generated by the multilingual sentence-transformer embedding model. Each chunk was successfully converted into a high-dimensional numerical vector capturing its full semantic meaning, which was subsequently indexed into the ChromaDB vector database to enable efficient cosine similarity search during retrieval. The result confirms that the embedding pipeline

correctly processed Amharic legal text and produced meaningful vector representations ready for semantic search operations.

4.4 Model Building

The model building phase focused on translating the system design into a fully functional RAG-based Amharic legal assistant by implementing the complete pipeline from embedding and indexing through to response generation. The cleaned document chunks were converted into dense vector embeddings using a multilingual sentence-transformer model from Hugging Face Transformers and stored in ChromaDB as the primary vector database, while in parallel a sparse BM25 index was constructed using the rank-bm25 library to capture exact term-level relevance signals. These two indexes formed the foundation of the hybrid retrieval mechanism, which combined dense semantic search, sparse keyword search, and Reciprocal Rank Fusion to maximize retrieval accuracy across diverse legal queries. The top-ranked retrieved chunks were subsequently passed to the GROQ API, which powered the large language model responsible for generating coherent and contextually grounded responses in Amharic. To prevent hallucinated responses when no relevant information was found, the system returned a predefined message "በተሰጠው ሕግ መረጃ ውስጥ ለዚህ ጥያቄ በቂ እና ትክክለኛ መረጃ አልተገኘም።" ("No sufficient and accurate information was found in the provided legal data for this question.") ensuring response reliability and factual grounding at all times.

The backend of the system was developed using FastAPI, providing a robust and scalable application programming interface for handling user queries and coordinating the retrieval and generation pipeline. The frontend chatbot interface was implemented using HTML, CSS, and JavaScript, delivering a clean, responsive, and Amharic-compatible conversational experience. Beyond core question answering functionality, the system incorporated user authentication, conversation history storage, and administrator document management capabilities, collectively forming a complete and deployable legal information assistant tailored to the needs of Amharic-speaking users in Ethiopia.

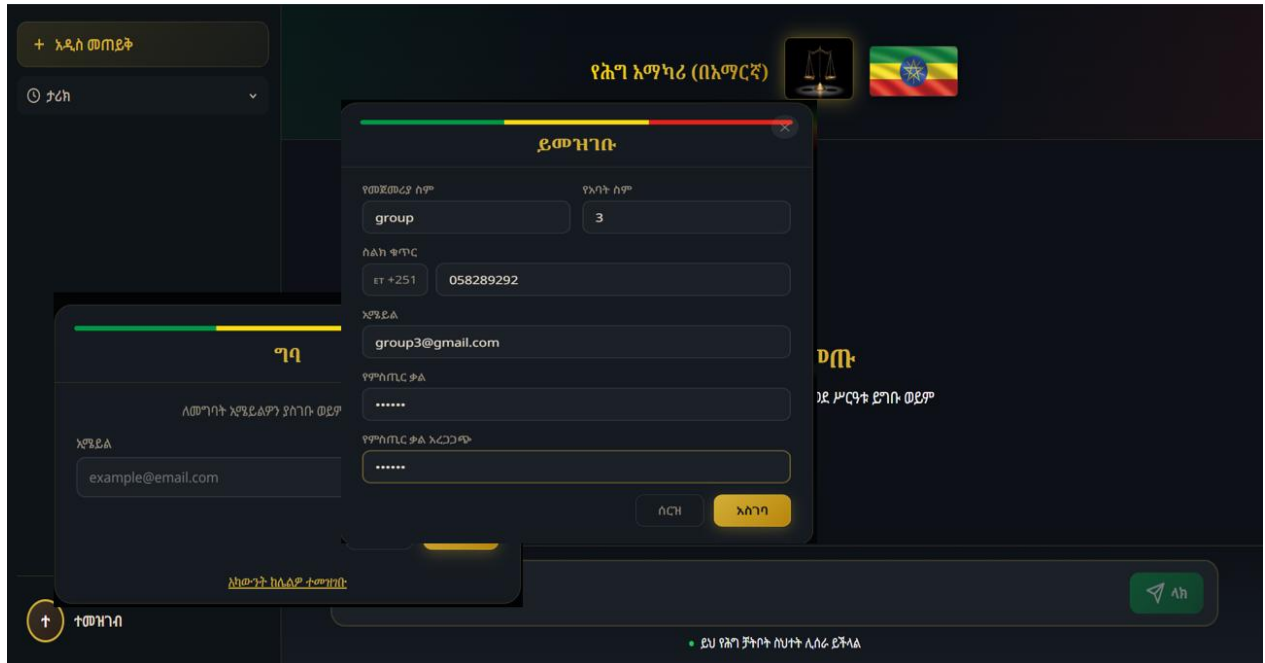


Figure 5 Sign Up Page

The figure 5 illustrates the user registration interface of the Amharic legal assistant system, allowing new users to create an account by providing their personal credentials to access the legal information service.

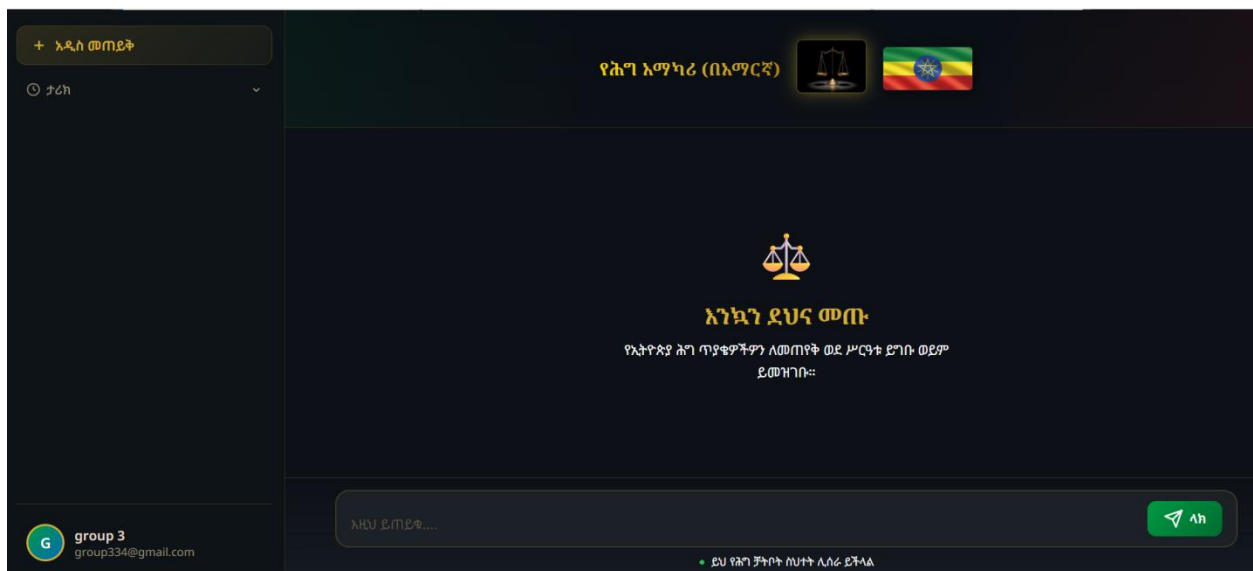


Figure 6 Chatbot Interface

The figure 6 illustrates the main conversational interface of the Amharic legal assistant system, where users can submit legal questions and receive contextually grounded responses in Amharic.

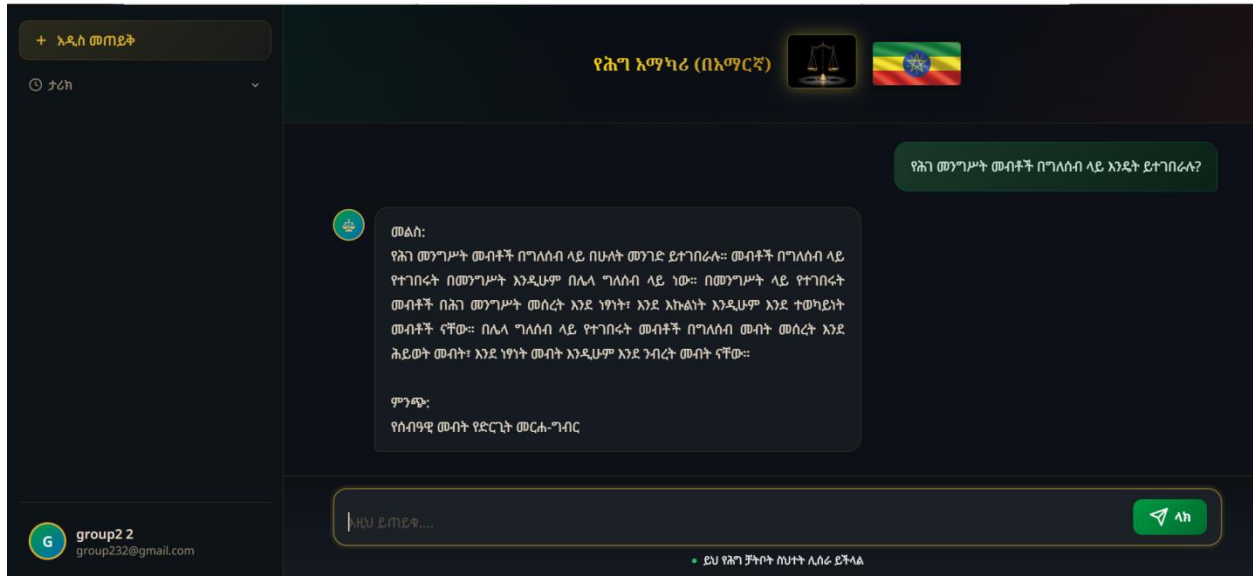


Figure 7 System Response When Relevant Data is Found

The figure 7 demonstrates the system successfully retrieving and generating a relevant legal response in Amharic when a user submits a query that matches the available information within the indexed document vector database.

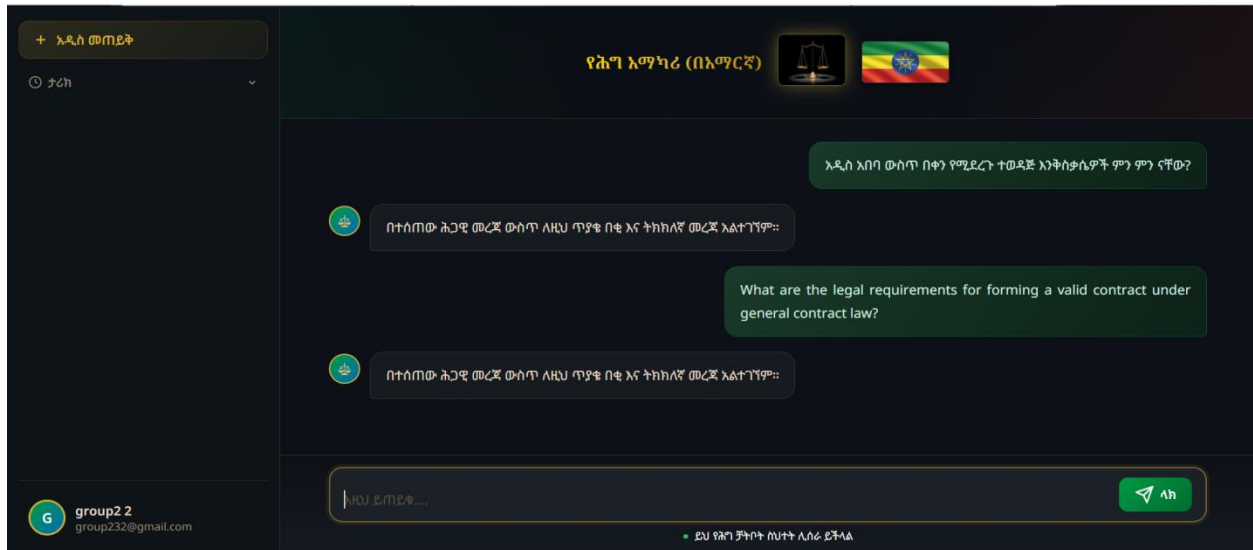


Figure 8 System Response When No Relevant Information is Found

The figure 8 demonstrates the system returning a predefined fallback message in Amharic when a user submits a query that has no matching information within the indexed document vector database.

4.5 Testing and Evaluation

The system was rigorously tested to ensure it met the defined requirements and delivered an effective user experience for Amharic-speaking legal information seekers. Testing involved evaluating the functionality of the user authentication process, the accuracy and relevance of legal responses generated by the system, and the overall usability of the chatbot interface.

Since no large-scale annotated Amharic legal dataset is currently available, offline evaluation was performed using ten sample Amharic legal queries covering topics across different legal domains. Four standard metrics were used to assess system performance: BLEU, ROUGE-L, Semantic Similarity, and Context Relevancy.

	A	B	C
1	Question	Question	Ground Truth
2	Q001	በድርጅት ላይ ተፈጻሚ የሚሆነው የንግድ ስራ ገቢ የግብር መጣኔ ስንት ነው?	በድርጅት ላይ ተፈጻሚ የሚሆነው የንግድ ስራ ገቢ የግብር መጣኔ ስላሳ በመቶ (30%) ነው ።
3	Q002	ግብር የሚከፈልበት የንግድ ስራ ገቢ እንዴት ይሰላል?	ከንግድ ስራ ኩነቶች ጠቅላላ የግብር ዓመቱ ገቢ ላይ ለግብር ከፋዩ የተፈቀዱት ወጪዎች ተቀናሽ ከተደረጉ በኋላ የሚገኘው የገቢ መጠን ነው ።
4	Q003	ለበን አድራጊነት ዓላማ የሚደረጉ ስጦታዎች ተቀናሽ የሚደረጉት ለማን ሲሰጡ ነው?	ለኢትዮጵያ የበን አድራጊነት ድርጅት፣ ለኢትዮጵያ ማህበር ወይም ለመንግስት ጥራ ምላሽ ለመስጠት የተደረገ ስጦታ ሲሆን ነው ።
5	Q004	አንድ ግብር ከፋይ የደረሰበትን ኪሳራ ለስንት ዓመታት ማሸጋገር ይችላል?	ኪሳራው ከተከሰተበት ዓመት መጨረሻ ጀምሮ ከሚቆጠሩ አምስት የግብር ዓመታት በላይ ማሸጋገር አይችልም ።
6	Q005	ዋጋቸው የሚቀንስ ሀብቶች የእርጅና ቅናሽ መታሰብ የሚጀምረው መቼ ነው?	ሀብቱ የንግድ ስራ ገቢውን ለማስገኘት ለአገልግሎት ዝግጁ ከሆነበትና አገልግሎት መስጠት ከጀመረበት ቀን እንስሱ ነው ።
7	Q006	ተቀናሽ የማይደረጉ ወጪዎች ውስጥ ሁለቱን ጥቀስ?	የግል ወጪ፣ ማንኛውንም ህግ በመጣስ የሚጣል የገንዘብ ቅጣት ወይም ካላ ተቀናሽ አይደረጉም ።
8	Q007	ለልማት ወይም ለሀገር ሉዓላዊነት ጥሪ የሚደረግ አርዳታ ከጠቅላላ ገቢው ስንት በመቶ ለግብር ከፋዩ የሚፈቀደው ጠቅላላ ተቀናሽ ከግብር ከፋዩ የግብር ዓመቱ ገቢ እስር በመቶ (10%) መብለጥ የለበትም ።	
9	Q008	የንግድ ስራ ገቢ የትኞቹን ገቢዎች ይጨምራል?	ኢቃዎችን በማስተላለፍና አገልግሎቶችን በመስጠት የሚገኝ ገቢን እንዲሁም የንግድ ስራ ሀብትን በማስተላለፍ የሚገኝ የገንዘብ መጠንን ይጨምራል ።
10	Q009	በብድር ላይ የሚከፈል ወለድ እንደ ወጪ የሚታዘው መቼ ነው?	ወላዱ በብሄራዊ ባንክ እና በንግድ ባንኮች መካከል ካለው የወለድ መጣኔ በሁለት በመቶ (2%) የማይበልጥ ከሆነ ነው ።
11	Q010	ግዙፋዊ ሀልዎት የሌላቸው የንግድ ስራ ሀብቶች የሚበሉት ምን ዓይነት ሀብቶች ናቸው የቅጅ መሰብት፣ ፓተንት፣ ዲዛይን፣ የንግድ ምልክት ወይም ለተወሰነ ዘመን የሚያገለግሉ ተመሳሳይ ሀብቶች ናቸው ።	

Figure 9 evaluation dataset

The above figure 9 shows a prepared evaluation dataset containing sample Amharic legal questions paired with their corresponding ground truth answers, used to evaluate the performance of the developed RAG-based Amharic legal assistant system.

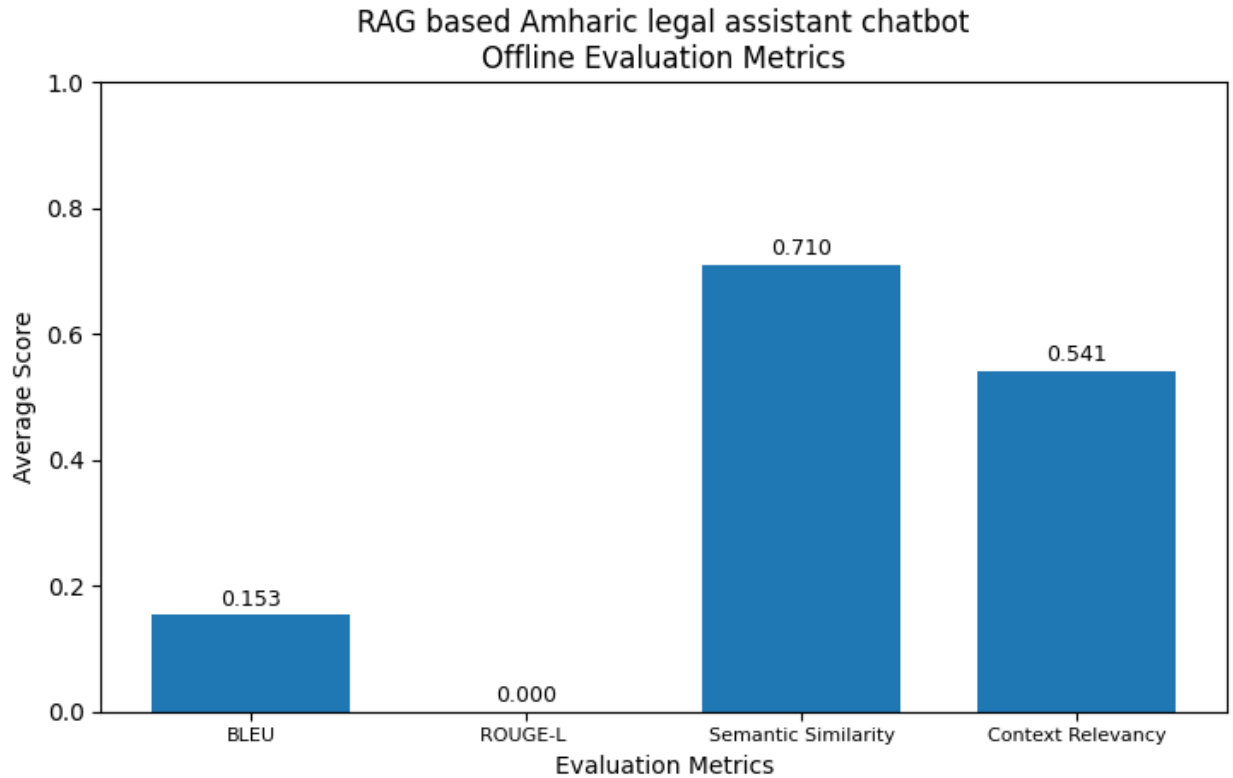


Figure 10 Offline Evaluation Metrics Bar Chart

The evaluation results demonstrated that the system effectively retrieved relevant legal information and generated accurate Amharic responses, with the Semantic Similarity score of 0.710 confirming strong semantic alignment between generated and expected answers despite limited exact word overlap. The low BLEU score of 0.153 and ROUGE-L score of 0.000 are consistent with the morphological complexity of Amharic rather than a failure in generation quality, while the system correctly returned a predefined fallback message when no relevant information was found in the knowledge base.

The Context Relevancy score of 0.541 reflects moderate retrieval performance, indicating that the hybrid retrieval mechanism improved coverage over single-strategy approaches, though further refinement of chunking strategies and embedding models is needed to enhance retrieval precision.

Therefore, building a RAG-based legal assistant for a low-resource language like Amharic presents significant challenges, yet the developed system proved capable of delivering strong semantic understanding and reliable response generation across diverse legal queries. The system integrated

a complete pipeline encompassing document chunking, vector embedding, hybrid retrieval, and large language model-based generation, each component contributing meaningfully to the overall performance. The evaluation results confirmed that the hybrid retrieval approach and contextually grounded generation pipeline collectively produced a functional and deployable system, while the fallback mechanism ensured response reliability when relevant information was unavailable. Overall, the chapter findings validate that the proposed system is a promising step toward making legal information more accessible to Amharic-speaking users in Ethiopia.

CHAPTER FIVE

CONCLUSION AND RECOMMENDATION

5.1 Conclusion

This study successfully designed and implemented a Retrieval-Augmented Generation Based Amharic Legal Document Assistant using a Large Language Model. The system was built to address the significant challenge of accessing legal information in Ethiopia, where legal documents are complex, technically worded, and largely inaccessible to the majority of Amharic-speaking citizens.

The system demonstrated the ability to retrieve relevant legal document chunks from Ethiopian criminal law and bail law documents and generate contextually grounded responses in Amharic based on the retrieved content. The hybrid retrieval mechanism combining dense semantic search, sparse BM25 retrieval, and Reciprocal Rank Fusion proved effective in improving retrieval coverage. The system also demonstrated correct fallback behavior by returning "በተሰጠው ሕጋዊ መረጃ ውስጥ ለዚህ ጥያቄ በቂ እና ትክክለኛ መረጃ አልተገኘም።" ("No sufficient and accurate information was found in the provided legal data for this question") when no relevant information was found in the knowledge base, preventing hallucinated responses.

The evaluation conducted using ten manually prepared sample Amharic legal queries in CSV format showed a Semantic Similarity score of 0.710, confirming that the system consistently produces semantically meaningful responses in Amharic. The low BLEU score of 0.153 and ROUGE-L score of 0.000 are consistent with the known limitations of token-overlap metrics when applied to morphologically rich low-resource languages such as Amharic. The Context Relevancy score of 0.541 reflected moderate retrieval performance with room for further improvement. Overall, this work establishes a solid foundation for the development of more advanced intelligent legal assistant systems tailored to the Ethiopian legal context.

5.2 Recommendation

Future development of the system should prioritize expanding language support to include other Ethiopian languages such as Oromiffa, Tigrinya, and Somali to broaden accessibility beyond

Amharic-speaking users. The system should also be transitioned from a research prototype into a fully deployed real-world application with a user-friendly interface designed to accommodate low-literacy and less technologically experienced users. Additionally, investing in building a comprehensive and well-structured Amharic legal dataset covering all major domains of Ethiopian law would significantly improve retrieval accuracy and system evaluation reliability. Finally, future work should explore fine-tuning a multilingual transformer model on a domain-specific Amharic legal dataset to reduce dependency on the GROQ API, enable offline operation, and produce more legally precise responses tailored to the Ethiopian legal context.

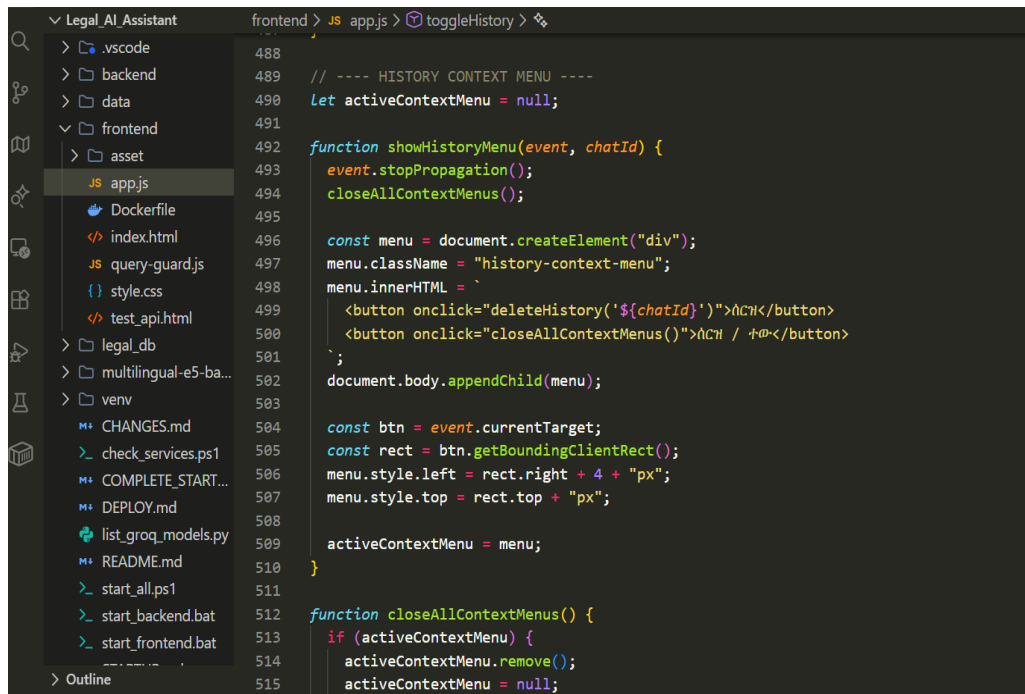
References

- [1] D. Jurafsky and J. H. Martin, **Speech and Language Processing**, 3rd ed. Stanford, CA, USA: Stanford University, 2021.
- [2] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in **Proc. NAACL-HLT**, Minneapolis, MN, USA, 2019, pp. 4171–4186.
- [3] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in **Advances in Neural Information Processing Systems (NeurIPS)**, vol. 33, 2020, pp. 9459–9474.
- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in **Proc. EMNLP**, Hong Kong, China, 2019, pp. 3982–3992.
- [5] A. Conneau et al., “Unsupervised Cross-lingual Representation Learning at Scale,” in **Proc. ACL**, Online, 2020, pp. 8440–8451.
- [6] OpenAI, “Introducing ChatGPT,” Nov. 29, 2022. [Online]. Available: <https://openai.com/index/chatgpt> [Accessed: Apr. 2026].
- [7] LegalZoom, “LegalZoom.” [Online]. Available: <https://www.legalzoom.com> [Accessed: Apr. 2026].
- [8] MWakili, “MWakili - Wakili AI.” [Online]. Available: <https://mwakili.com/> [Accessed: Apr. 2026].
- [9] Ministry of Justice, Federal Democratic Republic of Ethiopia, “Amharic Legal Documents.” [Online]. Available: <https://justice.gov.et/am/resource-tag/amharic-legal-documents/> [Accessed: Apr. 2026].
- [10] Abyssinia Law, “Civil Law Forms.” [Online]. Available: <https://www.abysiniaw.com/civil-law-forms?layout=table> [Accessed: Apr. 2026].

[11] 5A Law Firm, “Codes.” [Online]. Available: <https://5alawfirm.com/codes> [Accessed: Apr. 2026].

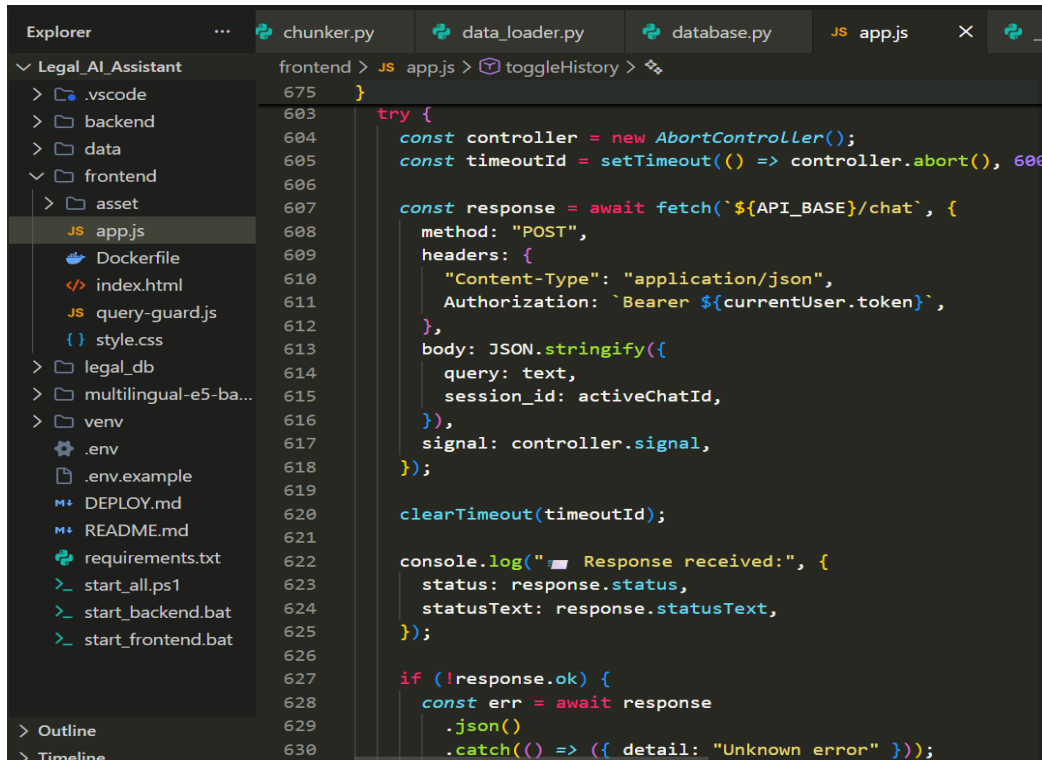
Appendices

Appendix A: User registration funsssssctionality



```
488
489 // ---- HISTORY CONTEXT MENU ----
490 let activeContextMenu = null;
491
492 function showHistoryMenu(event, chatId) {
493   event.stopPropagation();
494   closeAllContextMenus();
495
496   const menu = document.createElement("div");
497   menu.className = "history-context-menu";
498   menu.innerHTML = `
499     <button onclick="deleteHistory('${chatId}')">Delete</button>
500     <button onclick="closeAllContextMenus()">Close</button>
501   `;
502   document.body.appendChild(menu);
503
504   const btn = event.currentTarget;
505   const rect = btn.getBoundingClientRect();
506   menu.style.left = rect.right + 4 + "px";
507   menu.style.top = rect.top + "px";
508
509   activeContextMenu = menu;
510 }
511
512 function closeAllContextMenus() {
513   if (activeContextMenu) {
514     activeContextMenu.remove();
515     activeContextMenu = null;
```

Appendix B: Response generation functionality



```
675 }
603 try {
604   const controller = new AbortController();
605   const timeoutId = setTimeout(() => controller.abort(), 600
606
607   const response = await fetch(`${API_BASE}/chat`, {
608     method: "POST",
609     headers: {
610       "Content-Type": "application/json",
611       Authorization: `Bearer ${currentUser.token}`,
612     },
613     body: JSON.stringify({
614       query: text,
615       session_id: activeChatId,
616     }),
617     signal: controller.signal,
618   });
619   clearTimeout(timeoutId);
620
621   console.log("■ Response received:", {
622     status: response.status,
623     statusText: response.statusText,
624   });
625
626   if (!response.ok) {
627     const err = await response
628       .json()
629       .catch(() => ({ detail: "Unknown error" }));
630
```